AirInk

A Python Finger-Tracking Whiteboard

Project Report

Thiyatira Reni N
RA2511003010637
SRM Institute of Science and Technology, Kattankulathur

1. Introduction

AirInk is a desktop application that lets a user write in the air with a webcam. It tracks the index fingertip and draws its path on the screen. Raising only the index finger starts writing. Raising two fingers pauses it, so the user can move to the next stroke.

The drawing can appear over the camera image or on a whiteboard. A small side panel provides colours, brush size, an eraser, clear, save and reopen controls. Mouse drawing is available when the camera is off.

2. Objectives

- Build a simple, touch-free drawing demonstration using Python.
- Use a clear finger pose to separate writing from movement.
- Keep the application code in one file that is easy to explain.
- Save and reopen local drawings; handle camera failures safely.

3. Features

- Index-finger writing; two-finger pause.
- Six colours, adjustable brush size and eraser.
- Camera zoom and opacity, plus whiteboard view.
- Local PNG storage, reopen, clear and mouse fallback.

4. Technologies Used

Technology	Purpose
Python 3.12	Application language
Tkinter / ttk	Window, buttons, slider and image display
MediaPipe Hand Landmarker	Detecting the hand and 21 landmarks
OpenCV	Camera input, mirroring, resizing and line drawing
NumPy	Ink, mask and background arrays
Pillow	Tkinter image conversion and PNG export

The tested dependencies are pinned in requirements.txt. The hand model is included in assets/. Camera frames are processed locally. After installing the dependencies, the application does not require an internet connection.

5. Existing Approach vs Proposed Application

Mouse / touch drawing	AirInk
A mouse, stylus or touch screen controls the pen.	A webcam observes the user's finger.
Clicking or touching starts a stroke.	A finger pose starts a stroke.
Input is generally more precise.	Movement and lighting affect precision.
Direct pointer input is familiar.	Offers a simple touch-free demonstration.

6. Project Structure

File	Purpose
main.py	Complete application: tracking, drawing and UI
run.py	Installs dependencies and launches the app
requirements.txt	Pinned Python dependencies
assets/hand_landmarker.task	Bundled hand-tracking model
README.md	Setup and usage

The main app uses one AirInk class. A separate launcher handles dependency installation and starts the application.

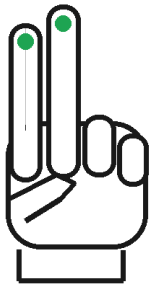
7. How the Project Works

1. Open the app and click Start camera.
2. OpenCV reads the full frame, mirrors it and converts it to RGB.
3. MediaPipe locates one hand and returns its landmarks.
4. The app checks whether only the index finger is raised.
5. The fingertip is mapped into the zoomed crop; three valid frames start drawing.
6. Consecutive points are joined on a separate ink layer.
7. The ink is shown over the camera image or a white background.
8. Save stores a PNG locally; Open restores a saved drawing to continue.

8. Finger-Pose Rules



Only index raised: draw / erase



Index + middle raised: pause

The palm must face the camera with the fingers pointing upward. The thumb must remain folded. Any pose other than index-only pauses writing. Losing the hand also pauses and clears the previous point.

For each finger: raised = fingertip.y < middle_joint.y − 0.025. The small margin reduces accidental activation when a finger is almost flat. The thumb is considered open when its tip is over 1.15 times as far from the little-finger base as its preceding joint.

For a crop (left, top, width, height), x = (tip.x − left) / width × 959 and y = (tip.y − top) / height × 539. Points outside the crop pause writing. Averaging smooths motion; jumps over 120 pixels start a new stroke.

9. Main Functions

Function	What it does
writing_pose()	Checks that only the index finger is raised
build_ui()	Creates the window and drawing controls
toggle_camera() / stop_camera()	Starts or releases the camera and tracker
tick()	Reads a frame, checks the pose and updates the screen
camera_view() / view_point()	Crops the display and maps fingertip coordinates
draw()	Smooths and connects points, or erases ink
compose()	Combines ink with the chosen background
save() / open_drawing()	Stores PNGs locally and restores editable ink
clear()	Clears the canvas
close()	Cancels the loop, releases resources and closes

The core idea

```
point = view_point(hand[8])
if point is not None and writing_pose(hand):
    draw(point)
else:
    previous = None
```

This is simplified pseudocode. The actual loop also waits for three valid frames, handles camera errors and draws the fingertip cursor.

Two image layers

The ink array stores RGB stroke colours. A separate mask records which pixels contain ink. Camera opacity blends only the camera with white (0–100%); the ink remains solid and tracking uses the original frame. Zoom crops only the display (1×–2.5×); tracking still sees the full frame. Erasing clears the mask, so the camera or whiteboard becomes visible again.

10. Screenshots

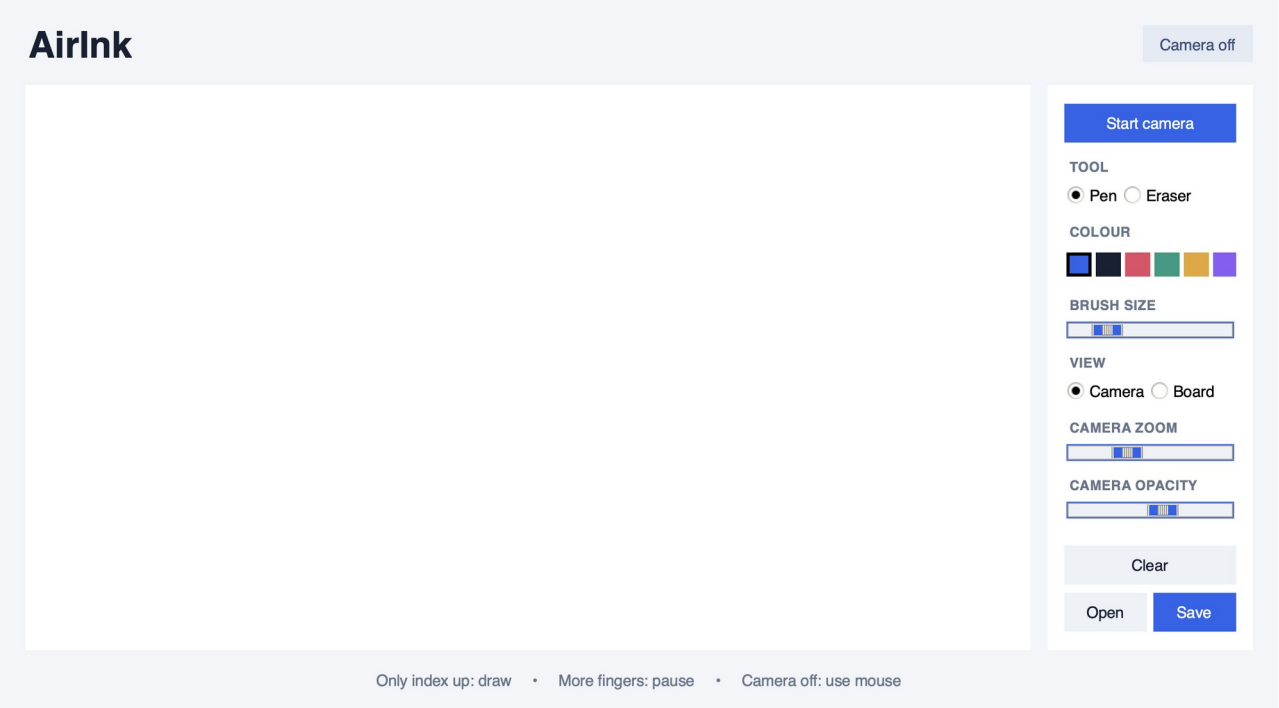


Figure 1. Actual application interface with the camera off.

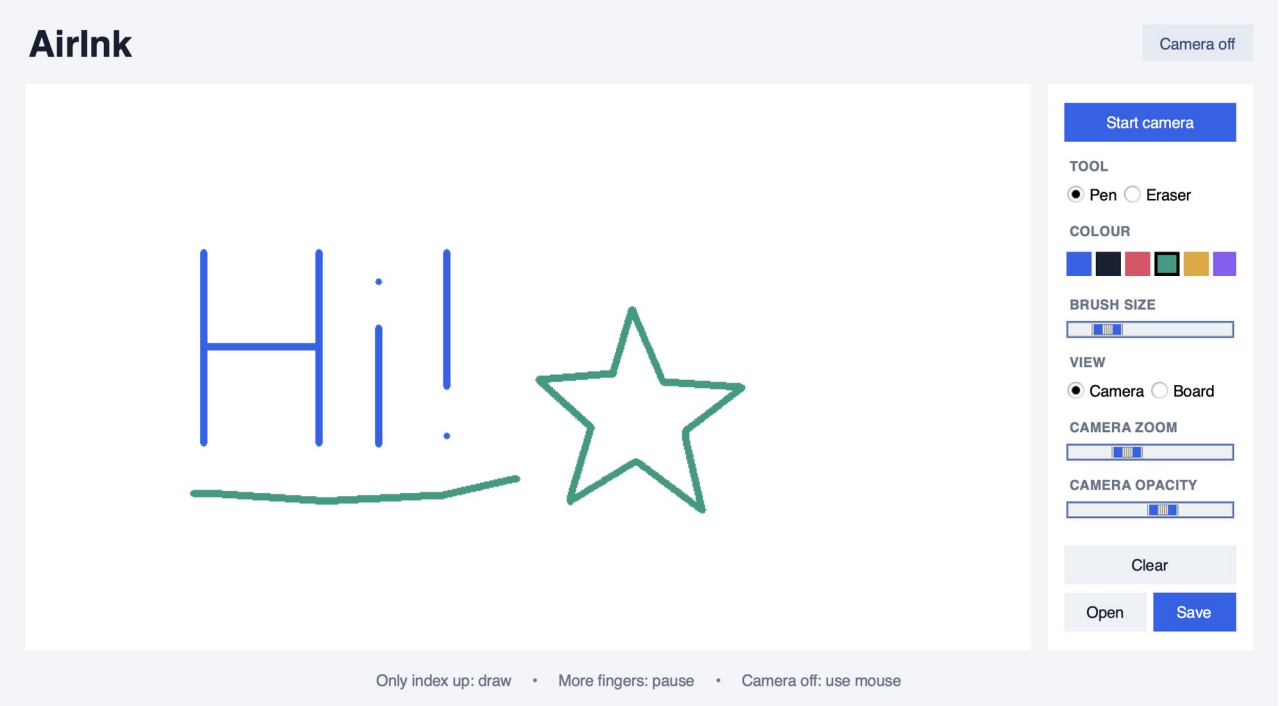


Figure 2. Actual interface with scripted sample strokes passed through draw().

These captures demonstrate the interface and drawing layer. They do not show live webcam tracking. Save stores only the drawing, without the panel or camera image.

11. Testing

All 38 automated tests passed on macOS with Python 3.12. The checks used the actual application functions and simulated camera responses where needed.

Area checked	Verified result
32 finger combinations; thumb and pose margin	Only index-only activates; nearly flat fingers pause
Ink colour and background	Correct RGB strokes appear on white
Camera opacity at 0%, 50% and 100%	Camera fades; ink stays solid; tracking receives the original frame
Display zoom and fingertip mapping	Full-frame tracking; correct crop, cursor and pause outside the crop
Pause and large movement	Separate strokes are not joined
Eraser and clear	Ink is removed and the background is restored
Colour change and PNG round trip	Pen switches colour; image retains the drawing
Save, reopen and continue	PNG survives app restart; drawing and erasing continue
Invalid file, cancelled open and failed save	Canvas is preserved; failures show an error
Platform paths and setup launcher	Mac/Windows paths, dependency reuse and installation failure checked
Camera start, disconnect and stop	Resources are released and mouse mode remains available
Three-frame activation and lost hand	Drawing starts after stability and pauses on tracking loss

Model check: the bundled MediaPipe model loaded successfully and detected one hand with 21 landmarks in MediaPipe's official thumb_up.jpg sample. An empty frame returned no hands.

Live checks still required

Webcam permission, lighting, tracking speed and finger pose need live checks. Windows paths were simulated; native Windows installation, launch and file dialogs still need checking. No measured live-camera accuracy or frame-rate claim is made.

12. Limitations and Future Work

Limitations

- The simple pose rule expects an upright, front-facing hand.
- Poor light, hidden fingers and motion blur can interrupt tracking. Zoom cannot recover a hand outside the physical camera frame.
- Tracking jitter and a brief three-frame delay affect handwriting.
- Only one hand is tracked; a mouse or stylus is more precise.
- Clear removes all ink. There is no undo or automatic saving.
- The project draws strokes; it does not recognise handwriting or control other applications.

Future work

- Add undo and redo.
- Use joint angles for gestures that work when the hand rotates.
- Add optional handwriting recognition and more export formats.

13. Conclusion

AirInk combines hand tracking, image processing and a small desktop interface in a single Python application. A simple finger pose controls when writing starts and stops. The project demonstrates camera input, condition checking, drawing, event handling and file export without a large codebase.

References and design credits

MediaPipe Hand Landmarker: ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/python

OpenCV Python documentation: docs.opencv.org/4.x/

Tkinter documentation: docs.python.org/3/library/tkinter.html

Pillow documentation: pillow.readthedocs.io/

Model and test image: Google MediaPipe assets (hand_landmarker.task and thumb_up.jpg).

Report structure and presentation scope: supplied TypeSprint examples. All AirInk content and screenshots describe this project.